

CAPSTONE PROJECT

DOCUMENTATION

End-to-End Automated Quality Assurance of ParaBank Service
Using Playwright Framework (JavaScript)

1. Project Overview & Scope

This capstone project delivers a robust, highly stable, and parallelized automation testing framework for the ParaBank financial services application (Target URL: <https://parabank.parasoft.com/>). Built from scratch using Playwright with JavaScript (JS), the framework implements industry-standard QA methodologies including the Page Object Model (POM), cross-browser isolation testing, and backend API verification. The scope covers exactly 8 micro-banking services with 15 test cases each, generating exactly 120 total automated test variations to ensure comprehensive regressions without execution flakiness.

2. Project Technical Stack

- **Language:** JavaScript (ES6+ Standards)
- **Automation Framework:** Playwright Test Runner
- **Target Application URL:** <https://parabank.parasoft.com/>
- **Design Pattern:** Page Object Model (POM) in JavaScript
- **Reporting:** Playwright Built-in HTML & Allure Report
- **CI/CD Integration:** Jenkins Integration Complete

3. Complete 120 Banking Test Cases Matrix (8 Services x 15 Test Cases)

Service 1: User Authentication & Onboarding

Test ID	Test Scenario Description	Expected Result	Syllabus / Concept Mapping
TC-AUTH-01	Verify new customer registration with entire sequence of valid field parameters.	Account created successfully, welcome page dashboard is loaded.	getByRole(), Hard Assertion
TC-AUTH-02	Verify registration validation block when mandatory inputs like SSN or Address are missing.	Field specific validation warning labels are highlighted.	Soft Assertion, Validation Check

TC-AUTH-03	Verify registration system prevents duplicate username creation entries.	Error banner 'This username already exists' is safely triggered.	Negative Testing Scenarios
TC-AUTH-04	Verify customer login using exact valid credential pairs.	Session initialized, redirected to internal account summary layout.	Page Fixture Lifecycle Hooks
TC-AUTH-05	Verify login failure processing when utilizing an absolute invalid password parameter.	Error message 'The username and password could not be verified' arrives.	Negative Testing Rules
TC-AUTH-06	Verify system state on trying to execute login with a blank username input.	Front-end validation enforces entry requirement constraints.	Security Input Bounds
TC-AUTH-07	Verify password masking configuration across the onboarding registration form structure.	Character streams render as secure bullet structures safely.	Input Box Configuration Check
TC-AUTH-08	Verify state preservation when refreshing the browser during active authenticated session rules.	User context remains active without forcing structural re-login.	Browser Context Reuse
TC-AUTH-09	Verify logout command flushes session context parameters and invalidates current states.	Redirected to index login block; back clicks cannot re-entry records.	Hooks, test.only Execution
TC-AUTH-10	Verify special character alphanumeric handling inside username parameters during registration blocks.	Sanitization schemas capture data patterns cleanly without errors.	Edge Case Input Handling

TC-AUTH-11	Verify basic accessibility role elements across the core homepage login panel interface.	All layout fields comply with standard aria-role configurations.	getByRole() Deep Dive
TC-AUTH-12	Verify login interface blocks standard basic SQL injection code string sequences gracefully.	System handles data payload strings as raw text parameters cleanly.	Security Element Validations
TC-AUTH-13	Verify password validation fields enforce character matching rules during onboarding.	Discrepancies trigger instant validation mismatched warning banners.	Dynamic Form Assertions
TC-AUTH-14	Verify leading and trailing white-spaces are stripped cleanly from login parameters.	Trims entries; processes valid matching sessions seamlessly.	Data Sanitization Blocks
TC-AUTH-15	Verify account response behavior on rapid automated sequence of invalid login submittals.	Infrastructure processing holds limits or triggers warning messages.	Rate Limit Verifications

Service 2: New Account Creation Service

Test ID	Test Scenario Description	Expected Result	Syllabus / Concept Mapping
TC-ACC-01	Verify opening a new structural Savings account via type selector dropdown entries.	New unique account index created with specific structural parameters.	Dropdown Select Automation
TC-ACC-02	Verify opening a new structural Checking account via type selector dropdown entries.	New checking instance successfully bound to current customer record.	UI Interaction Elements

TC-ACC-03	Verify primary funding source account selector list populates valid existing accounts.	Dropdown lists current accessible fund accounts accurately.	Parent-Child Traversal
TC-ACC-04	Verify new account initialization mandates minimum balance threshold transfers successfully.	System automatically moves initial funds to activate new ledger.	Functional State Verification
TC-ACC-05	Verify success screen outputs explicit valid new account ID link indicators.	Hyperlinked account ID is extracted and verified from DOM layout.	Text Value Extraction
TC-ACC-06	Verify clicking the generated new account link opens account details page layout.	Interface redirects seamlessly to exact item view of that account.	UI Navigation Sequence
TC-ACC-07	Verify initial deposit transactions match initial settings records precisely.	Ledger displays proper automated starting activity markers.	Data Verifications
TC-ACC-08	Verify system blocks account setup if funding sources possess zero balance balances.	Error warning triggers indicating insufficient startup funds available.	Boundary Assertions
TC-ACC-09	Verify rapid double execution of create account command processes a single instance.	UI temporarily toggles button state preventing dynamic duplicate records.	Click Rate Controls
TC-ACC-10	Verify accessibility aria-attributes on new account confirmation page elements.	Elements match getByRole semantic search validation checks cleanly.	Accessibility Target Logic
TC-ACC-11	Verify cross-browser layout visual correctness of the	Snapshot captures structure; layout	Visual Testing Rules

	account setup success view.	elements confirm styling properties.	
TC-ACC-12	Verify backend database updates immediately upon finishing UI creation workflow loops.	Account index immediately available in system general query lists.	State Management Testing
TC-ACC-13	Verify non-select dynamic item overlays close instantly when clicking escape keys.	Dropdown focus drops cleanly returning context to page bodies.	Keyboard Action Practices
TC-ACC-14	Verify processing behavior when account initialization endpoint returns timeout patterns.	System triggers standard loading spinner without layout crash loops.	Auto-waiting & Timeouts
TC-ACC-15	Verify minimum boundary transfer values are enforced on initial startup deposits.	Sub-dollar entries drop validation errors or enforce parameters.	Boundary Value Matrices

Service 3: Financial Dashboard & Accounts Overview (Advanced Tables)

Test ID	Test Scenario Description	Expected Result	Syllabus / Concept Mapping
TC-DASH-01	Verify account overview dashboard fetches and prints entire active ledger table grids.	Accounts table loads dynamically displaying all user indices.	Web Table Automation Basic
TC-DASH-02	Verify dynamic verification of row metrics against master user account indexes.	Table row count matches database records for current customer profile.	Advanced Table Row Loops
TC-DASH-03	Verify total balance column sum mathematically aggregates to final grand balance values.	UI values sum up matches printed summary total perfectly.	Data-Driven Assertions

TC-DASH-04	Verify extracting specific ledger cell text strings using parent-child element traversals.	Specific target data cell isolated and verified cleanly via script.	Parent-Child Traversal
TC-DASH-05	Verify clicking individual account number string redirects to transaction records.	Renders details, showing matching balances and historical rows.	Table Hyperlink Interaction
TC-DASH-06	Verify column sorting behaviors across the balance ledger data grid tables.	Click triggers sorting rearrangement matching selected parameter limits.	Table Automation Advanced
TC-DASH-07	Verify empty state visibility configurations if a profile possesses no asset indexes.	Presents structured clear notice stating 'No active accounts found'.	Conditional Layout Checks
TC-DASH-08	Verify visual masking verification across the balance overview table layouts.	Secures selected items, running screenshot matches safely.	Visual Testing & Masking
TC-DASH-09	Verify balance sheet updates instantly after executing secondary transaction operations.	Table values drop/increase to reflect external adjustments.	State Integration Testing
TC-DASH-10	Verify floating formatting structures across column currency parameter expressions.	All decimal fields display proper standard monetary symbols consistently.	Text Extraction Formats
TC-DASH-11	Verify table headers remain perfectly locked and visible during heavy scrolling operations.	CSS attributes verify view-port placement constraints seamlessly.	Style Attribute Validations
TC-DASH-12	Verify soft assertion validations across individual table record column rows.	Validates row data pairs without breaking entire test script workflows.	Soft Assertions Strategy

TC-DASH-13	Verify accessibility tags match grid semantic parameters on overview tables.	Table elements fulfill standard matrix reader roles perfectly.	getByRole() Verification
TC-DASH-14	Verify auto-waiting configuration isolates rendering lag across dashboard elements.	Framework holds step actions until data grids fully mount.	Auto-waiting Controls
TC-DASH-15	Verify negative balance data items display highlighted format styling cues.	Overdraft states show distinctive red text color styles in tables.	Dynamic CSS Validations

Service 4: Funds Transfer Service

Test ID	Test Scenario Description	Expected Result	Syllabus / Concept Mapping
TC-XFER-01	Verify valid asset transfer execution between two specific checking accounts.	Balance updates successfully; confirmation notice prints cleanly.	Functional Action Flow
TC-XFER-02	Verify blank value handling inside amount transfer input field structures.	Triggers inline prompt requesting a valid transfer parameter amount.	Form Validation Rules
TC-XFER-03	Verify system blocks transaction operations exceeding available account assets.	Error notifications indicate insufficient account funds present.	Boundary Asset Testing
TC-XFER-04	Verify dropdown account selection lists filter out inactive/closed index entities.	Only functional active bank indexes appear in transfer menus.	Dropdown Menus Filtering
TC-XFER-05	Verify entering negative numerical string values inside transaction amount inputs.	System invalidates request or forces absolute positive formatting.	Negative Scenario Input

TC-XFER-06	Verify fractional cent monetary entries calculate and process accurately.	System rounds according to policy rules or blocks entries.	Boundary Value Analysis
TC-XFER-07	Verify duplicate submission blocks on multi-clicking the action transfer control.	Button enters disabled state, ensuring single ledger debit execution.	Click Protection Controls
TC-XFER-08	Verify transfer details automatically log identical matching rows in history grids.	Both sender and receiver statements show corresponding transaction line rows.	Data Integration Checks
TC-XFER-09	Verify clearing fields resets drop menus back to default system indexes.	Forms clean layout parameters seamlessly on resetting operations.	UI Element Actions
TC-XFER-10	Verify accessibility tracking parameters across the funds movement form layout.	Input controls validate properly via getByRole configurations.	Accessibility Role Tracking
TC-XFER-11	Verify system behavior when transfer transaction back-end records experience timeouts.	Graceful handling displays retry flags without interface crash loops.	Timeouts Configuration
TC-XFER-12	Verify character strings are blocked from entry inside amount form parameters.	Restricts key input streams, preventing alphabetic characters.	Keyboard Action Rules
TC-XFER-13	Verify tracking trace viewer capture across entire fund movement workflows.	Generates complete diagnostic zip files containing telemetry assets.	Trace Viewer Grouping
TC-XFER-14	Verify soft validation matching across overview panels post transfer execution.	Checks balance states across different layouts concurrently.	Soft Assertions Practice

TC-XFER-15	Verify layout rendering stability on mobile resolution viewport parameters.	Forms resize cleanly, maintaining structural locator positions.	Project Configurations Options
-------------------	---	---	--------------------------------

Service 5: Bill Payment Service (Bill Pay)

Test ID	Test Scenario Description	Expected Result	Syllabus / Concept Mapping
TC-BP-01	Verify complete bill payout execution to a newly defined payee entity.	Transaction finishes smoothly; allocation message displays values.	Complex Form Automation
TC-BP-02	Verify validation rules blocking payout submissions when payee name inputs are blank.	Highlights missing parameters with clear input block warning messages.	Form Input Validations
TC-BP-03	Verify formatting constraints enforce strict zip-code syntax validations.	Malformed non-numeric zip sequences raise active validation errors.	Character Rule Testing
TC-BP-04	Verify account confirmation parameter checks mandate double account number matching entries.	Mismatched target index inputs trigger immediate form submission block.	Verification Form Match
TC-BP-05	Verify system validation blocks bill payments when using alphabetic amount parameters.	Form interface blocks entry streams or prints validation notices.	Input Box Constraints
TC-BP-06	Verify funding source drop menu accurate ledger calculations on selection shifts.	Altering source selection adjusts internal payment configuration data hooks.	Dropdown Automation Logic

TC-BP-07	Verify financial data updates immediately across ledger records post payout completion.	Debits target accounts instantly, recording specific bill-pay markers.	State Integration Checks
TC-BP-08	Verify visual styling matching of bill payment layout schemas via snapshot tools.	Confirms button placement, input rows alignment match spec patterns.	Visual Testing Rules
TC-BP-09	Verify boundary checks enforce maximum transaction limit rules on daily bill payouts.	Exceeding daily system limits triggers security blocking patterns.	Boundary Value Matrices
TC-BP-10	Verify special symbol parsing inside address fields behaves gracefully.	Processes complex strings containing characters like '#, -, /' properly.	Clean Data Inputs
TC-BP-11	Verify clearing form entries flushes current values completely across all inputs.	Returns fields to clean placeholders without requiring manual delete scripts.	UI Reset Automation
TC-BP-12	Verify focus tracking moves sequentially across form blocks utilizing tab keys.	Focus shifts correctly down form layout structures systematically.	Keyboard Action Practices
TC-BP-13	Verify accessibility attributes meet getByRole guidelines across bill payment fields.	Inputs map clean labels for automated assistive tracking scans.	Accessibility Role Testing
TC-BP-14	Verify telemetry reporting captures step timelines during long server responses.	Records system actions cleanly inside framework HTML report files.	Allure Reporting Controls
TC-BP-15	Verify system rejects payments where source indices contain zero assets.	Blocks execution flow, outputting 'Insufficient funds for operation' notices.	Negative Scenario Assertions

Service 6: Transaction Search & Financial Ledger

Test ID	Test Scenario Description	Expected Result	Syllabus / Concept Mapping
TC-SRCH-01	Verify search system filters ledger items utilizing valid transaction ID parameters.	Locates unique entry row, printing matching detail rows.	Parent-Child Traversal
TC-SRCH-02	Verify ledger query filtering behaves accurately using exact date parameters.	Table grid returns records restricted strictly to requested date bounds.	Web Table Data Search
TC-SRCH-03	Verify search index filters history records using specific monetary amount targets.	Isolates entries matching exact payment column values smoothly.	Extracting Values Match
TC-SRCH-04	Verify clear notification layout prints when search criteria returns empty rows.	Table grid prints standard message 'No transactions found for criteria'.	Negative Scenario Logic
TC-SRCH-05	Verify alpha character input blocking inside the transaction ID query fields.	Form inputs discard character strings or trigger inline warnings.	Input Box Data Controls
TC-SRCH-06	Verify clicking clear filters re-renders entire historical transaction table grid.	Resets overview layout, parsing complete history rows flawlessly.	UI Element Actions
TC-SRCH-07	Verify sorting order adjustments across filtered ledger table records.	Rearranges rows based on chronological or monetary sorting picks.	Table Automation Advanced
TC-SRCH-08	Verify extraction of transaction type labels	Validates row strings correctly classify debit versus credit actions.	Text Value Extraction

	from filtered result matrices.		
TC-SRCH-09	Verify boundary checks across wide range date interval query inputs.	Processes multi-month timeline parameter ranges cleanly without crash errors.	Boundary Value Analysis
TC-SRCH-10	Verify soft assertion loops validate multiple rows across filtering grids.	Validates complete data sets without interrupting remaining automation sequences.	Soft Assertions Rules
TC-SRCH-11	Verify accessibility roles follow grid guidelines on search results tables.	Verifies structure maps correctly under semantic getByRole queries.	Accessibility Grid Roles
TC-SRCH-12	Verify network request tracking captures transaction query response timelines.	Validates API payload speeds match performance constraints.	Trace Viewer Diagnostics
TC-SRCH-13	Verify calendar picker components parse dates cleanly without structural overlaps.	Handles dates parameters accurately, shifting focus back to inputs.	Complex Calendar Automation
TC-SRCH-14	Verify input boxes trim whitespaces from search keyword strings automatically.	Sanitizes parameter queries, delivering matching results seamlessly.	Data Sanitization Loops
TC-SRCH-15	Verify searching with completely invalid format strings returns standard notifications.	Handles bad queries safely, ensuring server does not return 500 errors.	Negative Assertions Engine

Service 7: Customer Support & Dispute Resolution

Test ID	Test Scenario Description	Expected Result	Syllabus / Concept Mapping
---------	---------------------------	-----------------	----------------------------

TC-SUPP-01	Verify Customer Care form accepts multi-line message block entries safely.	Textarea handles descriptive paragraphs without character loss errors.	Complex Form Automation
TC-SUPP-02	Verify form validation blocks support submissions when email inputs are blank.	Triggers error notice requiring email fields to be finalized.	Input Box Validations
TC-SUPP-03	Verify validation engine blocks invalid email string format parameters.	Detects syntax missing '@' or domain suffixes raising errors.	Syntax Boundary Checks
TC-SUPP-04	Verify file attachment feature handles single statement log document upload.	Path strings register cleanly, mounting file info into form arrays.	Single File Upload Concept
TC-SUPP-05	Verify uploading multiple error log documents simultaneously across inputs.	Array hooks map multiple file parameters successfully into UI states.	Multiple Files Upload Logic
TC-SUPP-06	Verify attachment file size controls block items exceeding platform rules.	System pops alert indicator preventing heavy file transfers.	Boundary Value Scenarios
TC-SUPP-07	Verify submittal of valid support forms pops confirmation dialog notices.	Browser intercept captures success alert panel notification windows.	Dialogs & Alerts Interception
TC-SUPP-08	Verify automated script dismisses/accepts support confirmation browser alerts.	Playwright script tracks modal, handling closing actions seamlessly.	Dialog Handling Core
TC-SUPP-09	Verify clear command flushes text strings cleanly across all fields.	Resets form layout back to original blank baseline states.	UI Element Actions

TC-SUPP-10	Verify accordion panels on support FAQ layouts expand on mouse clicks.	Click triggers expansion styling properties showing descriptive text blocks.	Accordion Component Automation
TC-SUPP-11	Verify support hyperlinks open external communication frameworks cleanly.	Clicks trigger target changes smoothly without dropping main session tabs.	Child Window Management
TC-SUPP-12	Verify accessibility role verification across core support interaction elements.	Fulfills standard assistive scanning expectations under getByRole.	Accessibility Role Tracking
TC-SUPP-13	Verify focus movement order maps predictably across support inputs using tabs.	Focus runs down sequential fields matching logical form structures.	Keyboard Action Tab flows
TC-SUPP-14	Verify running visual layout regressions across the customer care view panel.	Snapshots confirm element grid coordinates match blueprint properties.	Visual Testing & Masking
TC-SUPP-15	Verify input character limitations block data lengths exceeding boundaries.	Truncates massive paragraph streams safely matching configuration limits.	Boundary Assertions Engine

Service 8: Account Administration & Internal APIs

Test ID	Test Scenario Description	Expected Result	Syllabus / Concept Mapping
TC-API-01	Verify direct GET requests to customer account indexes endpoints fetch valid records.	Returns HTTP 200 status alongside complete profile account arrays.	Direct API Request Testing
TC-API-02	Verify JSON schema structures across internal API account overview outputs.	Payload parameters validate perfectly against master model data rules.	Schema Validation Check

TC-API-03	Verify GET endpoint queries passing specific transaction IDs map records accurately.	Returns target transaction data structures matching parameters precisely.	API Target Validations
TC-API-04	Verify GET requests using invalid non-existent account IDs trigger 404 metrics.	Server responds with standard HTTP 404 error payloads safely.	Negative API Scenarios
TC-API-05	Verify POST requests to internal transaction services map values cleanly.	Executes fund updates returning valid authorization transaction status flags.	API Transaction Flows
TC-API-06	Verify API intercept tools mock custom success payloads for external validations.	Playwright mock overrides network stream, outputting 200 simulation data.	API Interception + Mocking
TC-API-07	Verify network interception simulation handling server crash patterns gracefully.	Mocks HTTP 500 error payload, checking UI frontend alert reactions.	Network Interception Core
TC-API-08	Verify direct API configuration updates enforce state adjustments in database blocks.	Background parameters adjust system statuses verified by sequence loops.	State Integration Testing
TC-API-09	Verify data-driven automation loops testing multiple account data arrays.	Loops parameter matrices validating consistent behavior across sets.	Data-Driven Testing Scheme
TC-API-10	Verify missing authorization headers are caught blocking direct endpoint breaches.	Returns HTTP 401 response code ensuring backend data security.	Header Security Validations
TC-API-11	Verify system rejects API payloads containing corrupt	Backend drops transactions outputting	Payload Format Validations

	malformed JSON parameters.	HTTP 400 Bad Request statuses.	
TC-API-12	Verify transaction processing latency limits remain below system thresholds.	Requests finish processing actions within acceptable performance parameters.	API Performance Timing
TC-API-13	Verify API response content headers specify proper type constraints accurately.	Header properties enforce strict application/json return configurations safely.	Header Validation Engine
TC-API-14	Verify parameterized input sweeps executing direct endpoint queries across user indices.	Confirms systemic resilience running regressions across test data variations.	Parameterization Strategy
TC-API-15	Verify database state cleanup endpoints reset configuration variables cleanly.	Flushes simulation data rows restoring baseline account structures perfectly.	Backend Database Cleanup